



Figure 3.3: My Ancestor (all 5 root-to-leaf paths are sorted)

itself, we have found the solution. As there are Q queries, this approach runs in $O(QN)$ (the input tree can be a sorted linked list, or rope, of length N) and will get a TLE as $N \leq 80K$ and $Q \leq 20K$.

A better solution is to store all the $20K$ queries (we do not have to answer them immediately). Traverse the tree *just once* starting from the root using the $O(N)$ preorder tree traversal algorithm (Section 4.7.2). This preorder tree traversal is slightly modified to remember the partial root-to-current-vertex sequence as it executes. The array is always sorted because the vertices along the root-to-current-vertex path have increasing weights, see Figure 3.3 (right). The preorder tree traversal on the tree shown in Figure 3.3 (left) produces the following partial root-to-current-vertex sorted array: $\{\{3\}, \{3, 5\}, \{3, 5, 7\}, \{3, 5, 7, 8\}, \text{backtrack}, \{3, 5, 7, 9\}, \text{backtrack}, \text{backtrack}, \text{backtrack}, \{3, 8\}, \text{backtrack}, \{3, 6\}, \{3, 6, 20\}, \text{backtrack}, \{3, 6, 10\}, \text{and finally } \{3, 6, 10, 20\}, \text{backtrack}, \text{backtrack}, \text{backtrack (done)}\}$.

During the preorder traversal, when we land on a queried vertex, we can perform a $O(\log N)$ **binary search** (to be precise: **lower_bound**) on the partial root-to-current-vertex weight array to obtain the ancestor closest to the root with a value of at least P , recording these solutions. Finally, we can perform a simple $O(Q)$ iteration to output the results. The overall time complexity of this approach is $O(Q \log N)$, which is now manageable given the input bounds.

Bisection Method

We have discussed the applications of Binary Searches in finding items in static sorted sequences. However, the binary search **principle**⁴ can also be used to find the root of a function that may be difficult to compute directly.

Example: You buy a car with loan and now want to pay the loan in monthly installments of d dollars for m months. Suppose the value of the car is originally v dollars and the bank charges an interest rate of $i\%$ for any unpaid loan at the end of each month. What is the amount of money d that you must pay per month (to 2 digits after the decimal point)?

Suppose $d = 576.19$, $m = 2$, $v = 1000$, and $i = 10\%$. After one month, your debt becomes $1000 \times (1.1) - 576.19 = 523.81$. After two months, your debt becomes $523.81 \times (1.1) - 576.19 \approx 0$. If we are only given $m = 2$, $v = 1000$, and $i = 10\%$, how would we determine that $d = 576.19$? In other words, find the root d such that the debt payment function $f(d, m, v, i) \approx 0$.

An *easy* way to solve this root finding problem is to use the bisection method. We pick a reasonable range as a starting point. We want to fix d within the range $[a..b]$ where

⁴We use the term ‘binary search principle’ to refer to the D&C approach of halving the range of possible answers. The ‘binary search algorithm’ (finding index of an item in a sorted array), the ‘bisection method’ (finding the root of a function), and ‘binary search the answer’ (discussed in the next subsection) are all instances of this principle.

$a = 0.01$ as we have to pay at least one cent and $b = (1 + i\%) \times v$ as the earliest we can complete the payment is $m = 1$ if we pay exactly $(1 + i\%) \times v$ dollars after one month. In this example, $b = (1 + 0.1) \times 1000 = 1100.00$ dollars. For the bisection method to work⁵, we must ensure that the function values of the two extreme points in the initial Real range $[a..b]$, i.e. $f(a)$ and $f(b)$ have opposite signs (this is true for the computed **a** and **b** above).

a	b	d = $\frac{a+b}{2}$	status: $f(d, m, v, i)$	action
0.01	1100.00	550.005	undershoot by 54.9895	increase d
550.005	1100.00	825.0025	overshoot by 522.50525	decrease d
550.005	825.0025	687.50375	overshoot by 233.757875	decrease d
550.005	687.50375	618.754375	overshoot by 89.384187	decrease d
550.005	618.754375	584.379688	overshoot by 17.197344	decrease d
550.005	584.379688	567.192344	undershoot by 18.896078	increase d
567.192344	584.379688	575.786016	undershoot by 0.849366	increase d
...	...	...	a few iterations later ...	...
...	...	576.190476	stop; error is now less than ϵ	answer = 576.19

Table 3.1: Running Bisection Method on the Example Function

Notice that bisection method only requires $O(\log_2((b - a)/\epsilon))$ iterations to get an answer that is good enough (the error is smaller than the threshold error ϵ that we can tolerate). In this example, bisection method only takes $\log_2 1099.99/\epsilon$ tries. Using a small $\epsilon = 1\text{e-}9$, this yields only ≈ 40 iterations. Even if we use a smaller $\epsilon = 1\text{e-}15$, we will still only need ≈ 60 tries. Notice that the number of tries is *small*. The bisection method is much more efficient compared to exhaustively evaluating each possible value of $d = [0.01..1100.00]/\epsilon$ for this example function. Note: The bisection method can be written with a loop that tries the values of $d \approx 40$ to 60 times (see our implementation in the ‘binary search the answer’ discussion below).

Binary Search the Answer

The abridged version of UVa 11935 - Through the Desert is as follows: Imagine that you are an explorer trying to cross a desert. You use a jeep with a ‘large enough’ fuel tank – initially full. You encounter a series of events throughout your journey such as ‘drive (that consumes fuel)’, ‘experience gas leak (further reduces the amount of fuel left)’, ‘encounter gas station (allowing you to refuel to the original capacity of your jeep’s fuel tank)’, ‘encounter mechanic (fixes all leaks)’, or ‘reach goal (done)’. You need to determine the *smallest possible* fuel tank capacity for your jeep to be able to reach the goal. The answer must be precise to three digits after decimal point.

If we know the jeep’s fuel tank capacity, then this problem is just a simulation problem. From the start, we can simulate each event in order and determine if the goal can be reached without running out of fuel. The problem is that we do not know the jeep’s fuel tank capacity—this is the value that we are looking for.

From the problem description, we can compute that the range of possible answers is between $[0.000..10000.000]$, with 3 digits of precision. However, there are $10M$ such possibilities. Trying each value sequentially will get us a TLE verdict.

Fortunately, this problem has a property that we can exploit. Suppose that the correct answer is X . Setting your jeep’s fuel tank capacity to any value between $[0.000..X-0.001]$

⁵Note that the requirements for the bisection method (which uses the binary search principle) are slightly different from the binary search algorithm which needs a sorted array.

will *not* bring your jeep safely to the goal event. On the other hand, setting your jeep fuel tank volume to any value between $[X..10000.000]$ will bring your jeep safely to the goal event, usually with some fuel left. This property allows us to binary search the answer X ! We can use the following code to obtain the solution for this problem.

```
#define EPS 1e-9 // this value is adjustable; 1e-9 is usually small enough
bool can(double f) { // details of this simulation is omitted
    // return true if the jeep can reach goal state with fuel tank capacity f
    // return false otherwise
}

// inside int main()
// binary search the answer, then simulate
double lo = 0.0, hi = 10000.0, mid = 0.0, ans = 0.0;
while (fabs(hi - lo) > EPS) { // when the answer is not found yet
    mid = (lo + hi) / 2.0; // try the middle value
    if (can(mid)) { ans = mid; hi = mid; } // save the value, then continue
    else lo = mid;
}

printf("%.3lf\n", ans); // after the loop is over, we have the answer
```

Note that some programmers choose to use a constant number of refinement iterations instead of allowing the number of iterations to vary dynamically to avoid precision errors when testing $\text{fabs}(\text{hi} - \text{lo}) > \text{EPS}$ and thus being trapped in an infinite loop. The only changes required to implement this approach are shown below. The other parts of the code are the same as above.

```
double lo = 0.0, hi = 10000.0, mid = 0.0, ans = 0.0;
for (int i = 0; i < 50; i++) { // log2 ((10000.0 - 0.0) / 1e-9) ~ 43
    mid = (lo + hi) / 2.0; // looping 50 times should be precise enough
    if (can(mid)) { ans = mid; hi = mid; }
    else lo = mid;
}
```

Exercise 3.3.1.1: There is an alternative solution for UVa 11935 that does not use ‘binary search the answer’ technique. Can you spot it?

Exercise 3.3.1.2*: The example shown here involves binary-searching the answer where the answer is a floating point number. Modify the code to solve ‘binary search the answer’ problems where the answer lies in an *integer range*!

Remarks About Divide and Conquer in Programming Contests

The Divide and Conquer paradigm is usually utilized through popular algorithms that rely on it: Binary Search and its variants, Merge/Quick/Heap Sort, and data structures: Binary Search Tree, Heap, Segment Tree, Fenwick Tree, etc. However—based on our experience, we reckon that the most commonly used form of the Divide and Conquer paradigm in